

Complément d'informatique

TP 2 - Organisation de programmes

29 septembre 2026

L'objectif de cette séance de travaux dirigés est de vous familiariser avec le contenu de la partie 4 du cours théorique sur l'organisation de programme.

Comme pour les TPs précédents, des prototypes de code et des makefiles sont fournis pour chaque exercice pour vous faciliter le développement.

L'exercice 2 (généricité) est l'exercice à soumettre sur la plateforme de soumission de Montefiore et qui comptera pour 4% de la cote finale du cours. Pour être validé, votre code doit être soumis avant le vendredi **9/10/2025 à 23h59**. Cet exercice doit être réalisé seul et soumis individuellement par chaque étudiant.

Bon travail !

Exercice conseillé avant celui à soumettre

Exercice 1 (Pointeurs de fonctions):

1. Expliquez ce que fait le code suivant :

```
#include <stdio.h>

int f1(int (*f) (int));
int f2(int i);

int main(void) {
    printf("Answer: %d\n", f1(f2));
    return 0;
}

int f1(int (*f)(int)) {
    int n = 0;
    while ((*f)(n)) n++;
    return n;
}

int f2(int i) {
    return i*i+i-12;
}
```

Vérifiez votre réponse en exécutant le code.

(repris du livre “C programming : a modern approach”, K.N. King)

2. Ecrivez une fonction `int filter(int (* f)(int), int *array, int length)` retirant du tableau `array` de longueur `length` toutes les valeurs pour lesquelles `f(array[i])` renvoie une valeur différente de 0. Lorsqu’une valeur est retirée du tableau toutes les valeurs qui suivent dans le tableau doivent être décalée d’une position vers la gauche. La fonction doit renvoyer la longueur du tableau après filtrage (c’est-à-dire le nombre d’éléments du tableau n’ayant pas été retirés). Complétez ensuite la fonction `main` (fichier `main.c`) pour retirer au moyen de cette fonction les valeurs paires du tableau `myarray`.

Exercice à soumettre

Exercice 2 (Généricité):

On souhaite écrire une version générique d'une fonction permettant de trouver la première valeur d'un tableau qui respecte une certaine propriété, sous l'hypothèse que les éléments du tableau sont **triés** de sorte que les éléments ne respectant pas la propriété précèdent ceux qui respectent la propriété. Cette fonction appelée `findFirstTrue` aura la signature suivante :

```
int findFirstTrue(bool (* f)(void *, int, void *), void *tab, int length, void *param);
```

où `tab` est un tableau de longueur `length`, `f(tab, i, param)` est une fonction renvoyant `true` (respectivement, `false`) si la valeur à la position `i` du tableau respecte (respectivement, ne respecte pas) la propriété recherchée. L'argument `param` de `findFirstTrue` doit être passé tel quel en argument à `f`. Il permet de passer des paramètres supplémentaires à la fonction `f` si nécessaire. Si aucun élément du tableau ne respecte la propriété, la fonction devra renvoyer `length`.

Cette fonction doit avoir une complexité dans le pire cas $O(\log(n))$, où n est la longueur du tableau. Vous pouvez obtenir cette complexité en développant une solution inspirée de la recherche dichotomique.

Comme application de cette fonction, on vous demande ensuite d'écrire deux nouvelles fonctions :

- une fonction `int findLastPointinBall(Point *tab, int length, float r)` renvoyant le point le plus à droite du tableau `tab` dont la distance euclidienne au premier point du tableau est inférieure ou égale au rayon `r` en argument. On supposera que les points sont triés par ordre croissant de distance au premier point du tableau.
- Une fonction `int getPercentile(int *tab, int length, float p)` renvoyant la première position dans le tableau d'entiers `tab` strictement supérieure à $p \times \text{tab}[\text{length} - 1]$. Le tableau `tab` sera supposé trié par ordre croissant de valeurs.

Le fichier `main.c` vous permet de tester les trois fonctions. En particulier, la fonction `getPercentile` est testée sur une application concrète qui consiste à binariser une image en niveau gris. Le tableau `tab` représente le nombre cumulé de pixels de l'image pour chaque niveau de gris de 0 à 255 et la fonction `getPercentile` permet d'obtenir le niveau de gris tel qu'approximativement une proportion p (respectivement $1 - p$) des pixels de l'image soit en dessous (respectivement au dessus) de ce seuil. L'image est alors binarisée en remplaçant par 0 (noir) les pixels en dessous et par 255 (blanc) les pixels au dessus du seuil.

La solution de cet exercice est à rendre sur Gradescope et comptera pour 4% de votre cote finale. Soumettez **uniquement** le fichier `findFirsttrue.c` complété sur Gradescope.

Autres exercices

Exercice 3 (Masquage d'information et types opaques):

Le fichier `percolation.c` fournit une implémentation de l'algorithme de calcul de percolation où une structure est utilisée pour représenter une grille (plutôt qu'un simple tableau d'entiers comme c'était le cas dans le fichier disponible sur la page web du cours théorique). Isolez les fonctions de manipulation de la grille dans un module séparé, constitué de deux nouveaux fichiers `grid.h` et `grid.c`, en veillant à rendre la structure de la grille opaque. Modifiez ensuite les fonctions restantes du fichier `percolation.c` pour qu'elles utilisent ce module et adaptez en conséquence le fichier `Makefile` pour qu'il compile votre nouveau code.

Exercice 4 (Tests unitaires et débogage):

Le but de cet exercice est de vous faire écrire des tests unitaires permettant d'évaluer l'exactitude d'une fonction.

Soit la fonction suivante :

```
int search_pattern(char *pattern, char *message)
```

où `pattern` et `message` sont deux chaînes de caractères. Si le `pattern` apparaît au sein du `message`, cette fonction doit renvoyer l'indice de la première lettre (de la première occurrence) du `pattern` dans le `message`. Dans le cas contraire, la fonction doit renvoyer -1. On supposera que cette fonction est toujours appelée avec des chaînes de caractères bien formées, c'est-à-dire de longueurs non nulles et terminées par un caractère de fin de chaîne `\0`.

Nous vous demandons d'écrire dans `test_searchpattern.c` une fonction `int test_search_pattern(int (*search_pattern)(char *pattern, char *message))`, renvoyant 1 si la fonction `search_pattern` passée en argument est correcte (i.e. correspond bien à la définition ci-dessus) et 0 sinon. Une fonction de test devant être écrite en principe avant d'écrire la fonction à tester, vous ne devez pas utiliser d'implémentation de la fonction `search_pattern` pour implémenter cette fonction.

Pour tester votre fonction, la fonction `main` fournie l'applique à 9 soumissions réelles d'étudiants de l'année 2019-2020. Parmi ces 9 soumissions, les 2 premières sont correctes alors que les 7 autres sont incorrectes. Votre fonction devra donc renvoyer 1 pour les deux premiers codes et 0 pour les 7 autres. Comme d'habitude, vous pouvez tester votre code via la commande `make run`. Attention, il n'est pas facile de détecter tous les cas.

Bonus. Une fois votre fonction `test_searchpattern` correcte, écrivez votre version de la fonction `search_pattern` (sans regarder les solutions correctes fournies) et vérifiez qu'elle passe bien tous les tests effectués par `test_search_pattern`.